Light

GCSE Computer Science - Searching and Sorting Algorithms (Set 2)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

GCSE • OCR J277 • Grades 3—9 | Topic test | 30 min | 55 marks | Answered: 0 / 30

00:00 Start Pause Reset ← Back to Algorithms Searching Sorting

Multiple-choice topic test for **GCSE Computer Science (OCR J277)**. You may skip questions. When you submit, you'll see your score, a grade estimate, and worked explanations. **Tip:** use the stopwatch above to time yourself.

Q1 (1 mark)

When a linear search successfully finds an item, what should the algorithm typically return?

- ○ **A.** The algorithm should return the index position where the item was found, allowing the program to access or use the found item directly
- ○ **B.** The algorithm should return the entire list
- ○ **C.** The algorithm should return nothing
- ○ **D.** The algorithm should return a random number

Q2 (1 mark)

What value is commonly returned by a search algorithm when the target item is not found in the list?

- ○ **A.** The value 0, indicating the start of the list
- ○ **B.** The value -1, which is an impossible index position, clearly indicating 'not found'
- ○ **C.** The length of the list
- ○ **D.** The last item in the list

Q3 (1 mark)

In binary search, how is the middle index typically calculated when searching between index positions 'low' and 'high'?

- ○ **A.** middle = (low + high) / 2, using integer division to get a whole number index
- ○ **B.** middle = low - high
- ○ **C.** middle = low x high
- ○ **D.** middle = high - low + 1

Q4 (1 mark)

For a list of n items, what is the maximum number of passes bubble sort might need to fully sort the list?

- ○ **A.** Exactly 1 pass regardless of list size
- ○ **B.** Exactly n passes
- ○ **C.** At most n - 1 passes, because after each pass at least one more element is in its final position

- ○ **D.** Exactly n squared passes

Q5 (1 mark)

Merge sort is described as a 'divide-and-merge' algorithm. What does this mean?

- ○ **A.** The algorithm runs backwards through the list
- ○ **B.** The algorithm must be run multiple times by the user
- ○ **C.** The algorithm only works on lists that repeat values
- ○ **D.** The algorithm calls itself to solve smaller sub-problems, dividing the list repeatedly until reaching base cases of single elements

Q6 (1 mark)

When insertion sort places a new element into the sorted portion, how are existing elements typically moved?

- ○ **A.** Elements larger than the new element are shifted one position to the right to make space for insertion
- ○ **B.** Elements are deleted and recreated in new positions
- ○ **C.** All elements are randomly shuffled
- ○ **D.** Elements are moved to a separate temporary list

Q7 (1 mark)

Why is the number of comparisons used to measure search algorithm efficiency?

- ○ **A.** Comparisons are the only operation computers can perform
- ○ **B.** Other operations cannot be counted
- ○ **C.** Comparisons use the most electricity
- ○ **D.** Comparing elements is the fundamental operation that determines how quickly a search finds its target - more comparisons means more work

Q8 (1 mark)

In binary search, how does the algorithm determine that a target item is not in the list?

- ○ **A.** It checks a special 'not found' flag at the start of the list
- ○ **B.** Binary search cannot detect when items are not found
- ○ **C.** It compares the target to every element before concluding not found
- ○ **D.** When the search range becomes empty (low index exceeds high index), the target cannot exist in the list

Q9 (1 mark)

What is the purpose of a 'swapped' flag variable in optimised bubble sort?

- ○ **A.** To track whether any swaps occurred during a pass - if no swaps happened, the list is sorted and the algorithm can stop early
- ○ **B.** To count the total number of items in the list
- ○ **C.** To indicate which direction to sort (ascending or descending)

- ○ **D.** To mark which items have already been compared

Q10 (1 mark)

A 'stable' sorting algorithm maintains the relative order of items with equal values. Which scenario demonstrates why stability matters?

- ○ **A.** Sorting a list of unique identification numbers
- ○ **B.** Sorting the same list multiple times
- ○ **C.** Sorting items from largest to smallest instead of smallest to largest
- ○ **D.** Sorting student records first by surname, then by form group - stability ensures students with the same form group stay in surname order from the first sort

Q11 (1 mark)

What is the most efficient scenario scenario for insertion sort's performance?

- ○ **A.** When the list contains all identical values
- ○ **B.** When the list is in completely random order
- ○ **C.** When the list is already sorted in the desired order - each element only needs one comparison to confirm it's in the correct position
- ○ **D.** When the list is sorted in reverse order

Q12 (1 mark)

Why is linear search sometimes called 'sequential search'?

- ○ **A.** Because it checks items one after another in sequence, from the first to the last, following the order they appear in the list
- ○ **B.** Because it requires items to be numbered sequentially
- ○ **C.** Because it can only search sequences of numbers
- ○ **D.** Because it generates a sequence of random numbers

Q13 (2 marks)

Binary search is used to find the value 15 in the sorted list:
[2, 5, 8, 11, 15, 18, 21, 24]
Trace through the algorithm showing which elements are compared.

- ○ **A.** Compare 11 (middle), 15 > 11 so search right half; compare 18 (middle of right), 15 < 18 so search left of that; compare 15, found. Three comparisons total.
- ○ **B.** Compare 2, 5, 8, 11, 15 - five comparisons like linear search
- ○ **C.** Compare 15 directly - one comparison
- ○ **D.** The algorithm cannot find 15 in this list

Q14 (2 marks)

For a list of 5 items, how many comparisons does bubble sort make in the first pass?

- ○ **A.** 5 comparisons - one for each item
- ○ **B.** 1 comparison - only the first two items

- ○ **C.** 10 comparisons - comparing every item to every other item
- ○ **D.** 4 comparisons - comparing each adjacent pair (positions 0-1, 1-2, 2-3, 3-4)

Q15 (2 marks)

Two sorted sub-lists [2, 7, 9] and [3, 5, 8] are being merged. Show the sequence in which elements are added to the merged result.

- ○ **A.** 2, 7, 9, 3, 5, 8 - first list then second list
- ○ **B.** 3, 5, 8, 2, 7, 9 - second list then first list
- ○ **C.** 2, 3, 5, 7, 8, 9 - elements taken in order by comparing front elements of each sub-list and selecting the smaller
- ○ **D.** 9, 8, 7, 5, 3, 2 - largest elements first

Q16 (2 marks)

Insertion sort is being used on [4, 2, 5, 1]. After processing the third element (5), what does the list look like?

- ○ **A.** [4, 2, 5, 1] - unchanged because 5 is already in position
- ○ **B.** [2, 4, 5, 1] - the first three elements are now sorted among themselves
- ○ **C.** [1, 2, 4, 5] - the list is fully sorted
- ○ **D.** [5, 4, 2, 1] - sorted in descending order

Q17 (2 marks)

A phonebook app stores 10,000 contacts in alphabetical order. When a user types a name to search, which approach is most efficient?

- ○ **A.** Delete all contacts and re-enter them one by one until the name is found
- ○ **B.** Sort the contacts first, then search
- ○ **C.** Linear search - it's simpler to implement
- ○ **D.** Binary search - since contacts are alphabetically sorted, binary search can find any name in about 14 comparisons maximum, compared to up to 10,000 for linear search

Q18 (2 marks)

What input arrangement represents the least efficient scenario for bubble sort when sorting in ascending order?

- ○ **A.** Already sorted in ascending order - [1, 2, 3, 4, 5]
- ○ **B.** Random order - [3, 1, 4, 2, 5]
- ○ **C.** Sorted in descending order - [5, 4, 3, 2, 1] - every element must bubble through all positions to reach its correct place, requiring maximum swaps
- ○ **D.** All identical values - [3, 3, 3, 3, 3]

Q19 (2 marks)

Why does merge sort have higher memory requirements than bubble sort?

- ○ **A.** Merge sort stores the original list multiple times for backup
- ○ **B.** During the merge process, merge sort creates new arrays to hold the merged results; it cannot merge two sub-lists directly back

into the original space without additional memory for temporary storage

- ○ **C.** Merge sort requires memory for counting comparisons
- ○ **D.** Bubble sort compresses data to use less memory

Q20 (2 marks)

A teacher wants to find all students who scored exactly 75 marks in a class of 30 students (unsorted data). What modification to linear search is needed?

- ○ **A.** No modification - linear search automatically finds all matches
- ○ **B.** Instead of stopping when a match is found, continue searching through the entire list and record all matching positions in a separate list or array
- ○ **C.** Sort the data first, then use binary search
- ○ **D.** Search the list backwards instead of forwards

Q21 (2 marks)

A music app needs to sort a user's playlist of 20 songs. The playlist changes frequently as users add and remove songs. Which sorting approach is most practical?

- ○ **A.** Never sort the playlist - display songs in random order
- ○ **B.** Use merge sort every time a song is added
- ○ **C.** Use insertion sort - for small lists it's efficient, and when adding a single song to an already-sorted playlist, insertion sort can add it in the correct position with minimal work

○ **D.** Delete the entire playlist and recreate it whenever changes are made

Q22 (2 marks)

In a binary search implementation, what happens if the comparison operators are accidentally reversed (using > instead of <)?

○ **A.** The algorithm works exactly the same

○ **B.** The algorithm searches in the wrong half after each comparison, likely failing to find items that exist or returning incorrect positions

○ **C.** The algorithm sorts the data instead of searching

○ **D.** The algorithm runs twice as fast

Q23 (2 marks)

Compare insertion sort and bubble sort for sorting a list of 10 items that is already sorted except for one item at the wrong position.

○ **A.** Both algorithms perform identically on all inputs

○ **B.** Bubble sort would be faster because it's simpler

○ **C.** Insertion sort is likely faster - it can place the mispositioned item correctly by shifting only the necessary elements, while bubble sort might need multiple passes to fully reposition the item

○ **D.** Neither algorithm can handle partially sorted data

Q24 (3 marks)

Analyse the following linear search implementation for a function that returns the index of a target in an array, or -1 if not found:

```
function linearSearch(array, target)
 for i = 0 to array.length
 if array[i] == target then
 return i
 endif
 next i
 return -1
endfunction
```

Identify any errors and explain the correct implementation.

- ○ **A.** The implementation is completely correct with no errors
- ○ **B.** The loop should use array.length - 1 as the upper bound because array indices go from 0 to length-1. Using array.length would attempt to access an index beyond the array bounds, causing an error. The corrected loop: for i = 0 to array.length - 1
- ○ **C.** The function should return the target value, not the index i
- ○ **D.** Linear search cannot be implemented as a function

Q25 (3 marks)

Binary search is used to find the value 42 in this sorted list:
[10, 20, 30, 40, 50, 60, 70, 80]
Trace through the algorithm completely and determine the outcome.

- ○ **A.** 42 is found at index 4
- ○ **B.** 42 is found at index 3
- ○ **C.** The search determines 42 is not in the list after checking indices 3, 5, and 4 in sequence. The search range narrows to empty when looking between 40 and 50, confirming 42 doesn't exist in this list of multiples of 10.

○ **D.** Binary search cannot determine if an item is missing

Q26 (3 marks)

Analyse the total number of comparisons made by optimised bubble sort (with early termination) versus unoptimised bubble sort when sorting an already-sorted list of 100 items.

○ **A.** Both versions make the same number of comparisons regardless of input

○ **B.** Unoptimised: 99 + 98 + 97 + ... + 1 = 4,950 comparisons across all passes. Optimised: 99 comparisons in one pass, detects no swaps, terminates immediately. The optimisation provides a 50x improvement for already-sorted input.

○ **C.** Optimised makes more comparisons due to the overhead of checking the swap flag

○ **D.** Neither version works on sorted input

Q27 (3 marks)

Trace the complete merge sort process on [7, 2, 6, 3], showing all divide and merge steps.

○ **A.** The list is too small for merge sort to work correctly

○ **B.** Divide: [7,2,6,3] gives [7,2] [6,3] gives [7] [2] [6] [3]. Merge: [2,7] [3,6] gives [2,3,6,7]. The algorithm divides until single elements, then merges pairs in sorted order, producing the final sorted result.

○ **C.** [7,2,6,3] gives [2,3,6,7] in one step with no intermediate stages

○ **D.** Merge sort cannot sort lists with an even number of elements

Q28 (3 marks)

A hospital emergency department needs to sort incoming patients by urgency level (1-5 scale, where 1 is most urgent). Patients arrive continuously, and the sorted list must always be current.
Analyse which sorting approach is most appropriate.

- ○ **A.** Sort all patients at midnight each day using any algorithm
- ○ **B.** Random ordering is acceptable since all patients are eventually treated
- ○ **C.** Use bubble sort on the entire list every time a patient arrives
- ○ **D.** Use insertion sort - as each new patient arrives, insert them into the correct position in the already-sorted list. This maintains real-time sorting with minimal operations per insertion. For a list of n patients, adding one patient requires at most n comparisons, far better than re-sorting the entire list.

Q29 (3 marks)

An online game stores a leaderboard of 100,000 player scores, sorted from highest to lowest. The game needs to: (1) quickly check if a score qualifies for the leaderboard, and (2) display a player's rank.
Analyse the most efficient approach.

- ○ **A.** Use linear search for both operations since it's simplest to implement
- ○ **B.** For checking qualification: compare against the lowest leaderboard score (position 99,999) - one comparison determines if the new score beats it. For finding rank: use binary search on the sorted scores to find where the player's score would be inserted - this gives their rank in about 17 comparisons. Together, these exploit the sorted structure for maximum efficiency.
- ○ **C.** Sort the leaderboard fresh each time a score is submitted
- ○ **D.** Store scores unsorted and calculate ranks on demand by

counting

Q30 (3 marks)

Compare merge sort and bubble sort for sorting a large dataset of 10,000 randomly ordered items, considering both time efficiency and memory usage.

- ○ **A.** Bubble sort is better in all respects for large datasets
- ○ **B.** Both algorithms have identical performance on large datasets
- ○ **C.** Merge sort: approximately 130,000 comparisons (each item is compared about 13 times during the divide-and-merge process). Bubble sort: up to 50,000,000 comparisons least efficient scenario. Merge sort is roughly 380 times faster but requires extra memory for the sub-lists. For large datasets, merge sort's speed advantage far outweighs its memory cost.
- ○ **D.** Memory usage is the only difference between the algorithms

Total: **55** marks • Time: **30 min**

↑ Back to top

Submit answers

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.